



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE INGENIERÍA

DIVISION DE INGENIERIA ELECTRICA

DISEÑO DIGITAL MODERNO

Semestre 2027-1

Tarea 01.

Profesor: Dr. Oscar Francisco Fuentes
Casarrubias

Alumnos:

- Cabrera Luna Yael
- Pérez Hernández Carlos Alberto
- Reyes Alcántara Angel Gabriel
- Vargas Pérez Patricia Victoria

Grupo: 5

Fecha de entrega: 8 de septiembre de 2026



Antes de comenzar con la explicación de este código, debemos aclarar que la solución de esta tarea se realizó mediante el lenguaje de programa Python, esta elección fue tomada debido a que visualizamos que nos podría optimizar bastantes partes del código, y en efecto así fue, haciendo que la elaboración del mismo fuese más eficiente.

Problema 1: Conversor base numérica con validación

Para la resolución de este problema, primero decidimos crear una función, en la cual se le implementó todo el código necesario para en pasos posteriores solo mandarla a llamar y darle los parámetros necesarios.

Esta función se nombró “convertir bases” y recibirá un total de tres parámetros, el primero de ellos es la variable “basem” que representará la base original de cual el usuario va a partir, después recibirá “basen” que tendrá como información la base a la que se quiere convertir o viéndolo de otra forma, la base objetivo a la cual queremos llegar, y finalmente recibirá número, que no es más que el número de interés que se quiere transformar de base; esta última variable cuenta con la peculiaridad de guardarse como un string (cadena) para que nos sea más fácil trabajar con la parte entera y decimal.

```
def convertirbases(basem, basen, numero):
```

Ahora bien, definimos los símbolos posibles que serán permitidos en nuestro programa, para este caso serán los números del cero al nueve y todas las letras del abecedario, al guardarlos de la forma que se muestra a continuación se estaría asignando el valor correspondiente a la letra según la posición en la que se encuentre, es decir, la letra A tendrá el valor de 10 y así sucesivamente como lo vimos en clase.

```
alfabeto = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ"  
numfinalentero = ""  
numfinalfraccion = ""
```

Las variables “numfinalnombre” mostradas en la imagen anterior están guardando una cadena vacía, pues este valor se irá generando a lo largo de la ejecución del programa.

Para poder trabajar con mayor facilidad los números ingresados con parte decimal según corresponda, utilizamos la función “.partition”, esta función lo que hace es separar la cadena en tres partes, para nuestro problema dividirá la parte entera, el punto decimal y la parte decimal de nuestro numero (recordando que esta guardado como una cadena de texto).

```
numeroentero, punto, numerofraccion = numero.partition(".")
```

Después de haber declarado los puntos importantes antes mencionados, implementamos un ciclo for general que tendrá el trabajo de recorrer el número ingresado por el usuario.

```
for simbolo in revision:
```

La primera función de este ciclo for es verificar con ayuda de una condición if si el número ingresado contiene elementos del abecedario declarado, en caso de que no el programa arrojará un mensaje indicando al usuario que los símbolos ingresados no corresponden.

```
if simbolo not in alfabeto:  
    print("Error: El número contiene símbolos no registrados.")  
    validacion = False  
    break
```

Si la condición anterior no fue aplicada, entonces el programa sigue, y su siguiente acción declarar un variable llamada valor, esta variable tendrá como propósito buscar el valor numérico que le corresponde a las letras del abecedario como en un principio establecimos. Seguido de ello, para controlar los símbolos o bien números, que se pueden ingresar según la base origen, nos apoyamos de una condición if, la cual evaluará si el valor numérico de un x elemento es mayor o igual al valor de la base original, si se cumple entonces el programa imprimirá un

mensaje indicando al usuario que el caracter ingresado no pertenece a la base de origen que seleccionó.

```
if valor >= basem:  
    print("Error: El número contiene símbolos inválidos para la base original.")  
    validacion = False  
    break
```

Ya con las validaciones pertinentes que nos rectifican que el número ingresado corresponde a la base de origen y que dicho número contiene elementos del abecedario definido, la función se pasa a la conversión de bases, para ello podríamos “dividir”, los procesos en dos partes, la primera de ella es la operación de base la base original a la base 10 y posteriormente convertir este resultado obtenido hacia la base seleccionada por el usuario. Para ello, comenzando con la conversión de la parte entera hacia la base 10, se creó una variable llamada “elevado” a la cual se le asigna la cantidad de símbolos que contiene la parte entera del número; después mediante un ciclo for se recorre cada uno de los símbolos contenidos en la variable “numeroentero”. En cada iteración se obtiene nuevamente el valor numérico que corresponde al símbolo utilizando la función “.index()”; a continuación la variable “elevado” disminuye una unidad para poder obtener el exponente correspondiente de acuerdo a la posición que ocupa cada dígito del número. Esta conversión se realiza mediante el código:

```
potencia = valor*basem**(elevado)
```

Esta operación corresponde al procedimiento utilizado para convertir un número de cualquier base hacia 10, es decir, se multiplica cada dígito por la base original elevada a la potencia que le corresponde según su posición en la que se encuentre. Cada uno de los resultados obtenidos se acumula dentro de la variable “resultadoentero”, para poder obtener el valor decimal resultante de la parte entera.

Por otro lado, para la parte fraccionaria se realiza un procedimiento similar, pero solamente que ahora las potencias son negativas. Para comenzar, se reinicia la variable “elevado” con un valor de cero para después recorrer cada símbolo de la variable “numerofraccion”, en cada símbolo recorrido, el valor de la variable “elevado” aumenta una unidad y vuelve a realizar la operación:

```
potencia = valor*basem**(-elevado)
```

De este modo, el primer elemento de la parte fraccionaria utiliza una potencia de -1, el segundo una potencia de -2 y así sucesivamente; los resultados obtenidos se acumulan en la variable “resultadofraccion”.

Una vez que tanto la parte entera como la fraccionaria se encuentran convertidas en base 10, comienza la segunda conversión para transformar los resultados en la base solicitada por el usuario mediante la variable “basen”.

Para transformar la parte entera se utiliza un ciclo while el cual se estará ejecutando mientras el valor de “resultadoentero” sea mayor que cero. Dentro del ciclo se obtiene primero el residuo de dividir el número entre la base destino mediante el operador módulo (%); el residuo obtenido corresponder a uno de los símbolos del alfabeto definido anteriormente. Luego se realiza una división entera mediante el operador “//” el cual actualiza el valor de “resultadoentero”. El símbolo correspondiente al residuo se agrega al inicio de la cadena “numfinalentero” ya que se van obteniendo los residuos de manera inversa, este proceso se repite hasta que el cociente sea cero. Para convertir la parte fraccionaria se utiliza un ciclo for que se ejecutará cinco veces, esta cantidad de iteraciones se estableció con la finalidad de limitar el resultado a un máximo de cinco cifras después del punto decimal; además, en cada iteración la parte fraccionaria se multiplica por la base a la que se quiere transformar (elegida por el usuario), más tarde mediante “.partition(“.”)” se separa nuevamente la parte entera de la parte decimal del resultado obtenido.

La parte entera de esta multiplicación representa el siguiente dígito del número transformado, es por esto que se utiliza este valor como posición dentro de nuestro alfabeto y el símbolo correspondiente se agrega a “numfinalfraccion”. Una vez obtenido el símbolo solamente se conserva la parte decimal para repetir el mismo procedimiento durante la siguiente iteración; así se generan progresivamente los cinco dígitos correspondientes a la parte fraccionaria del número transformado.

Para finalizar, la función “convertirbases()” devuelve tres valores mediante la instrucción:

```
return numfinalentero, numfinalfraccion, validacion
```

que contiene la parte entera del número convertido y su parte decimal, sumando la validación del número, indicando si el número ingresado cumplió con las condiciones establecidas para realizar la conversión.

Problema 2: Emulador de Sumador Binario de 6 bits

Para la resolución del segundo problema se crearon dos funciones auxiliares, la primera de ellas es “a2()”, su propósito es trabajar con la representación binaria de los números tanto positivos como negativos, de este modo recibe dos parámetros: “binario” que corresponde al número que se desea trabajar y “signo” que permite conocer si el número original era positivo o negativo.

```
def a2(binario, signo):
```

Para empezar, se utiliza el método “.zfill(5)” para completar el número binario con ceros a la izquierda hasta alcanzar una longitud de cinco bits; por ejemplo si el número ingresado es “110”, al final se obtendría “00110”. A continuación, con la condición if se verifica el signo del número en la variable “signo”, si el valor es falso el número original es negativo y se tendrá que realizar algunas operaciones

para poder trabajar con el número. Para ello, primero se invierte la cadena mediante:

```
binarioinv = binario[::-1]
```

También se crea una cadena vacía “binc” en la cual se irá completando el nuevo número binario. La variable “volteo” es de tipo booleana para controlar el momento en el que se deberán invertir los bits; el ciclo for recorre cada uno de los bits empezando desde el lado derecho del número; de esta manera, mientras no se encuentre un primer “1”, los bits permanecen sin modificaciones. Una vez encontrado ese valor, los bits restantes se invertirán: los ceros se convierten en unos y los unos en ceros. Así, estas operaciones nos permite obtener la correcta representación del número negativo. Al terminar el ciclo, la cadena vuelve a invertirse para recuperar el orden original de los bits y finalmente la función devuelve el resultado.

Por otro lado, en caso de que el signo sea “True” el número será positivo y no es necesario aplicar este procedimiento y solamente se devuelve el número binario después de haber sido completado con los ceros correspondientes.

```
else:  
    binario = binario  
    return binario
```

La segunda función “suma_un_bit()” se encarga de realizar la suma de dos bits considerando el posible acarreo. Recibe tres parámetros: “bita”, “bitb” y “acarreo”; los primeros dos representan los bits que se desean sumar, mientras que el tercero almacena el posible acarreo generado anteriormente.

```
def suma_un_bit(bita, bitb, acarreo):
```

Para iniciar, los bits son transformados a valores enteros mediante y se realiza la suma de los tres valores almacenando el resultado en la variable "suma_con_acarreo". Luego, mediante el operador módulo %2 se obtiene el bit que deberá formar parte del resultado final, esto es debido a que una suma binaria solamente puede almacenar un cero o un uno en cada posición.

Después se verifica si la suma es mayor o igual que dos. Cuando esto ocurre, significa que debe generarse un acarreo hacia la siguiente posición, por lo que la variable "acarreo" toma el valor de uno, en caso contrario, mantiene el valor de cero.

Finalmente, la función devuelve tanto el bit resultante como el acarreo generado ya que ambos valores serán necesarios para continuar realizando la suma del resto del número.

```
return str(suma), acarreo
```

Una vez definidas las funciones, comienza la parte principal del programa: un menú principal implementado con un ciclo while para mantener su ejecución hasta que el usuario decida salirse.

El menú muestra un mensaje de bienvenida y tres opciones: realizar una conversión de base a base, la suma de números binarios o la salida del programa.

```
while True:
    print("\nBienvenido al programa de conversión de
    bases y suma de binarios.\n")
    print("Seleccione una opción:")
    print("1 = Conversion de base a base")
    print("2 = Suma de binarios")
    print("3 = Salir")
```


La opción seleccionada por el usuario se almacena dentro de la variable “op”. Cuando el usuario selecciona la opción número uno, el programa solicita tres datos: la base del número original, la base a la cual desea convertirlo y finalmente el número que desea transformar.

Estos valores son enviados como parámetros a la función “convertirbases()”; como anteriormente se mencionó, esta función devuelve tanto la parte entera como la parte fraccionaria del resultado, por lo que ambas son almacenadas dentro de las variables “numfinalentero” y “numfinalfraccion”. Finalmente, el programa muestra al usuario el número original, su base, el resultado obtenido y la nueva base.

```
print(f"El número {numero} en base {basem} es  
igual a {numfinalentero}.{numfinalfraccion} en base  
{basen}.\n")
```

Cuando el usuario selecciona la opción número dos, comienza el procedimiento correspondiente a la suma de números binarios. Primero se solicitan dos números enteros los cuales son almacenados dentro de las variables “numero1” y “numero2”. Después se crea una variable booleana “validacion”, la cual determina si los valores ingresados se encuentran dentro del rango permitido por el programa. Mediante dos condiciones se comprueba el valor de ambos números, si alguno se encuentra fuera del intervalo establecido se muestra un mensaje indicando que los valores están fuera del rango permitido y la variable booleana cambia a “False” evitando que continúe el procedimiento de suma. En cambio, si ambos números cumplen con las condiciones establecidas, el programa identifica el signo de cada uno; cuando un número es negativo, la variable correspondiente a su signo toma el valor de “False” y utiliza la función “abs()” para trabajar temporalmente con su valor absoluto, pero si el número es positivo, su variable de signo toma el valor “True”.

```

    validacion = True

    if(numero1 <= -32 or numero1 >= 31 ):
        print("Error: Los números ingresados están
fuera del rango permitido.")
        validacion = False

    elif(numero2 <= -32 or numero2 >= 31 ):
        print("Error: Los números ingresados están
fuera del rango permitido.")
        validacion = False

```

```

if validacion:
    if numero1 < 0:
        signo1 = False
        numero1 = abs(numero1)
    else:
        signo1 = True

    if numero2 < 0:
        signo2 = False
        numero2 = abs(numero2)
    else:
        signo2 = True

```

Una vez identificados los signos, ambos valores decimales son enviados nuevamente a la función “convertirbases()” utilizando en este caso base 10 como base original y base 2 como base a la cual se quiere transformar.

De esta forma se obtiene la representación binaria de ambos números y el resultado obtenido se almacena posteriormente dentro de las variables “binario1” y “binario2”; después ambos números son enviados a la función del complemento a2, la cual se encarga de completar los números hasta la cantidad de bits

establecida y cuando sea el caso generar la representación necesaria para los números negativos.

Una vez obtenidas las representaciones binarias, ambas cadenas son invertidas dado que la suma debe comenzar desde el bit ubicado más a la derecha, al igual que cuando realizamos una suma comenzando por las unidades. También, se crea una cadena vacía llamada “binariofinal”, en la cual se irán almacenando los bits resultantes, y una variable “acarreo” cuyo valor inicial es cero.

```
numfinalentero1, numfinalfraccion1 = convertirbases(10, 2, str(numero1))
numfinalentero2, numfinalfraccion2 = convertirbases(10, 2, str(numero2))

binario1 = numfinalentero1
binario2 = numfinalentero2

binario1 = a2(binario1, signo1)
binario2 = a2(binario2, signo2)

binario1v = binario1[::-1]
binario2v = binario2[::-1]
binariofinal = ""
acarreo = 0
```

Posteriormente, mediante la función “zip()”, se recorren simultáneamente los bits correspondientes de ambos números; cada par de bits es enviado a la función “suma_un_bit()” junto con el acarreo obtenido en la operación anterior. De este modo, la función devuelve el resultado de la suma del bit y el nuevo acarreo, el bit obtenido se agrega a la cadena “binariofinal” y este procedimiento continúa hasta haber recorrido completamente ambos números. Debido a que la suma fue realizada sobre las cadenas invertidas, al finalizar el procedimiento se vuelve a invertir, obteniendo de esta manera el resultado en su orden correcto.

```
for simbolo1, simbolo2 in zip(binario1v, binario2v):
    resultado, acarreo = suma_un_bit(simbolo1, simbolo2, acarreo)
    binariofinal = binariofinal + resultado

binariofinal = binariofinal[::-1]
```

El siguiente paso consiste en verificar si se produjo un desbordamiento u “overflow”. Este suceso ocurre cuando el resultado de una operación no puede ser representado utilizando la cantidad de bits disponibles. Para poder identificarlo, se crea una variable booleana “overflow” inicializada con “False”, después el programa analiza los signos de los números y el acarreo generado durante la suma para poder determinar si el resultado excedió la capacidad establecida. Si se detecta un desbordamiento, la variable booleana cambia a “True” y el programa muestra un mensaje de error al usuario. En caso de que no exista el desbordamiento el programa compara los valores de “numero1” y “numero2” considerando el signo que tenían desde un principio. Dependiendo de estos valores se utiliza la función “a2()” cuando es necesaria y se agrega un bit al inicio de la cadena con:

```
"1" + binariofinal
```

ó

```
"0" + binariofinal
```

Este primer bit indica el signo utilizado para interpretar posteriormente el resultado.

Una vez concluida la representación binaria final, se procede a obtener su equivalente decimal utilizando la función “convertirbases()” con la base 2 como base original y base 10 a la cual se quiere convertir. En la conversión, se lee la cadena desde la segunda posición, ya que el primer bit solo aporta el signo; asimismo, con una condición if si el primer bit corresponde a un “1” se agrega un signo negativo al resultado decimal.

```
if binariofinal[0] == "1":  
    decimal_resultado = "-" + decimal_resultado
```

Para concluir, el programa muestra tanto el resultado obtenido en formato binario como su representación equivalente en decimal.

```
if binariofinal[0] == "1":  
    decimal_resultado = "-" + decimal_resultado
```

La tercera opción del menú finaliza la ejecución del programa y mediante "break" se termina el ciclo while del menú principal.

```
elif op == 3:  
    print("Saliendo del programa...")  
    break
```

En último lugar, se implementó la condición para controlar cualquier opción diferente de las tres establecidas en el menú. Si el usuario ingresa un valor no válido, el programa muestra un mensaje indicando que debe seleccionar una opción correcta y se termina la ejecución.

```
else:  
    print("Opción inválida. Por favor, seleccione una opción válida.\n")  
    break
```

Pruebas y ejecuciones del programa.

Conversión de base 7 a base 2 fallida:

```
Bienvenido al programa de conversión de bases y suma de binarios.  
Seleccione una opción:  
1 = Conversión de base a base  
2 = Suma de binarios  
3 = Salir  
Su opción: 1  
Ingrese la base del número original: 7  
Ingrese la base del número nuevo: 2  
Ingrese el número a convertir: 17  
Error: El número contiene símbolos inválidos para la base original.
```

Conversión de base 16 a base 2 aceptada:

```
Bienvenido al programa de conversión de bases y suma de binarios.  
Seleccione una opción:  
1 = Conversion de base a base  
2 = Suma de binarios  
3 = Salir  
Su opción: 1  
Ingrese la base del número original: 16  
Ingrese la base del número nuevo: 2  
Ingrese el número a convertir: 1A  
El número 1A en base 16 es igual a 11010.00000 en base 2.
```

Suma de 2 números positivos:

```
Seleccione una opción:  
1 = Conversion de base a base  
2 = Suma de binarios  
3 = Salir  
Su opción: 2  
Ingrese su primer numero: 10  
Ingrese su segundo numero: 10  
  
El resultado de la suma es: 010100 en binario y 20 en decimal
```

Resta de 2 números:

```
Bienvenido al programa de conversión de bases y suma de binarios.  
Seleccione una opción:  
1 = Conversion de base a base  
2 = Suma de binarios  
3 = Salir  
Su opción: 2  
Ingrese su primer numero: 30  
Ingrese su segundo numero: -20  
  
El resultado de la suma es: 001010 en binario y 10 en decimal
```

Suma de 2 números negativos:

```
Bienvenido al programa de conversión de bases y suma de binarios.

Seleccione una opción:
1 = Conversion de base a base
2 = Suma de binarios
3 = Salir
Su opción: 2
Ingrese su primer numero: -20
Ingrese su segundo numero: -3

El resultado de la suma es: 110111 en binario y -23 en decimal
```

Suma con overflow:

```
Bienvenido al programa de conversión de bases y suma de binarios.

Seleccione una opción:
1 = Conversion de base a base
2 = Suma de binarios
3 = Salir
Su opción: 2
Ingrese su primer numero: 20
Ingrese su segundo numero: 20
Error: Se ha producido un desbordamiento (overflow) en la suma de los números binarios.
```

Más pruebas y ejecuciones del programa.

```
PS C:\Users\yaelc\AppData\Local\Programs\Microsoft VS Code> & C:\Users\yaelc\AppData\Local\Programs\Python\Python314\python.exe
Bienvenido al programa de conversión de bases y suma de binarios.

Seleccione una opción:
1 = Conversion de base a base
2 = Suma de binarios
3 = Salir
Su opción: 1
Ingrese la base del número original: 2
Ingrese la base del número nuevo: 16
Ingrese el número a convertir: 15
Error: El número contiene símbolos inválidos para la base original.

Bienvenido al programa de conversión de bases y suma de binarios.
```

```
Bienvenido al programa de conversión de bases y suma de binarios.

Seleccione una opción:
1 = Conversion de base a base
2 = Suma de binarios
3 = Salir
Su opción: 1
Ingrese la base del número original: 16
Ingrese la base del número nuevo: 2
Ingrese el número a convertir: F
El número F en base 16 es igual a 1111.00000 en base 2.

Bienvenido al programa de conversión de bases y suma de binarios.
```

```
Seleccione una opción:
1 = Conversion de base a base
2 = Suma de binarios
3 = Salir
Su opción: 1
Ingrese la base del número original: 2
Ingrese la base del número nuevo: 3
Ingrese el número a convertir: 11010
El número 11010 en base 2 es igual a 222.00000 en base 3.
```

Bienvenido al programa de conversión de bases y suma de binarios.

Seleccione una opción:

Bienvenido al programa de conversión de bases y suma de binarios.

```
Seleccione una opción:
1 = Conversion de base a base
2 = Suma de binarios
3 = Salir
Su opción: 2
Ingrese su primer numero: 300
Ingrese su segundo numero: -1
Error: Los números ingresados están fuera del rango permitido.
```

Bienvenido al programa de conversión de bases y suma de binarios.

```
Seleccione una opción:
1 = Conversion de base a base
2 = Suma de binarios
3 = Salir
Su opción: 2
Ingrese su primer numero: 31
Ingrese su segundo numero: 3
Error: Los números ingresados están fuera del rango permitido.
```

Bienvenido al programa de conversión de bases y suma de binarios.

```
Seleccione una opción:
1 = Conversion de base a base
2 = Suma de binarios
3 = Salir
Su opción: 2
Ingrese su primer numero: -300
Ingrese su segundo numero: -1
Error: Los números ingresados están fuera del rango permitido.
```


Bienvenido al programa de conversión de bases y suma de binarios.

Seleccione una opción:

1 = Conversion de base a base

2 = Suma de binarios

3 = Salir

Su opción: 2

Ingrese su primer numero: 21

Ingrese su segundo numero: -13

El resultado de la suma es: 001000 en binario y 8 en decimal

Bienvenido al programa de conversión de bases y suma de binarios.

Seleccione una opción:

1 = Conversion de base a base

2 = Suma de binarios

3 = Salir

Su opción: 2

Ingrese su primer numero: -13

Ingrese su segundo numero: -15

El resultado de la suma es: 111100 en binario y -28 en decimal

Bienvenido al programa de conversión de bases y suma de binarios.

Seleccione una opción:

1 = Conversion de base a base

2 = Suma de binarios

3 = Salir

Su opción: 1

Ingrese la base del número original: 16

Ingrese la base del número nuevo: 10

Ingrese el número a convertir: 3F

El número 3F en base 16 es igual a 63.00000 en base 10.

Bienvenido al programa de conversión de bases y suma de binarios.

Seleccione una opción:

1 = Conversion de base a base

2 = Suma de binarios

3 = Salir

Su opción: 1

Ingrese la base del número original: 8

Ingrese la base del número nuevo: 2

Ingrese el número a convertir: 56

El número 56 en base 8 es igual a 101110.00000 en base 2.

Bienvenido al programa de conversión de bases y suma de binarios.

Seleccione una opción:

1 = Conversion de base a base

2 = Suma de binarios

3 = Salir

Su opción: 1

Ingrese la base del número original: 2

Ingrese la base del número nuevo: 16

Ingrese el número a convertir: 110101

El número 110101 en base 2 es igual a 35.00000 en base 16.

Bienvenido al programa de conversión de bases y suma de binarios.

Seleccione una opción:

1 = Conversion de base a base

2 = Suma de binarios

3 = Salir

Su opción: 1

Ingrese la base del número original: 10

Ingrese la base del número nuevo: 8

Ingrese el número a convertir: 185

El número 185 en base 10 es igual a 271.00000 en base 8.

Bienvenido al programa de conversión de bases y suma de binarios.

Seleccione una opción:

1 = Conversion de base a base

2 = Suma de binarios

3 = Salir

Su opción: 1

Ingrese la base del número original: 4

Ingrese la base del número nuevo: 10

Ingrese el número a convertir: 213

El número 213 en base 4 es igual a 39.00000 en base 10.

Bienvenido al programa de conversión de bases y suma de binarios.

Seleccione una opción:

1 = Conversion de base a base

2 = Suma de binarios

3 = Salir

Su opción: 2

Ingrese su primer numero: 12

Ingrese su segundo numero: 11

El resultado de la suma es: 010111 en binario y 23 en decimal

Bienvenido al programa de conversión de bases y suma de binarios.

Seleccione una opción:

1 = Conversion de base a base

2 = Suma de binarios

3 = Salir

Su opción: 1

Ingrese la base del número original: 16

Ingrese la base del número nuevo: 10

Ingrese el número a convertir: 4B

El número 4B en base 16 es igual a 75.00000 en base 10.

Bienvenido al programa de conversión de bases y suma de binarios.

Seleccione una opción:

1 = Conversion de base a base

2 = Suma de binarios

3 = Salir

Su opción: 1

Ingrese la base del número original: 8

Ingrese la base del número nuevo: 2

Ingrese el número a convertir: 73

El número 73 en base 8 es igual a 111011.00000 en base 2.

Bienvenido al programa de conversión de bases y suma de binarios.

Seleccione una opción:

1 = Conversion de base a base

2 = Suma de binarios

3 = Salir

Su opción: 2

Ingrese su primer numero: 10

Ingrese su segundo numero: -1

El resultado de la suma es: 001001 en binario y 9 en decimal

Bienvenido al programa de conversión de bases y suma de binarios.

Seleccione una opción:

1 = Conversion de base a base

2 = Suma de binarios

3 = Salir

Su opción: 2

Ingrese su primer numero: -18

Ingrese su segundo numero: 8

El resultado de la suma es: 101010 en binario y -10 en decimal